Benchmarking Amplitude Estimation on a Superconducting Quantum Computer

Salvatore Certo,[1,*] Anh Dung Pham,[1] and Daniel Beaulieu[1]

[1] *Deloitte Consulting, LLP*

(Dated: January 28, 2022)

arXiv:2201.06987v2 [quant-ph] 27 Jan 2022

Amplitude Estimation (AE) is a critical subroutine in many quantum algorithms, allowing for a quadratic speedup in various applications like those involving estimating statistics of various functions as in financial Monte Carlo simulations. Much work has gone into devising methods to efficiently estimate the amplitude of a quantum state without expensive operations like the Quantum Fourier Transform (QFT), which is especially prohibitive given the constraints of current NISQ devices. Newer methods have reduced the number of operations required on a quantum computer and are the most promising near-term implementations of the AE subroutine. While it remains to be seen the exact circuit requirements for a quantum advantage in applications relying on AE, it is necessary to continue to benchmark the algorithm's performance on current quantum computers and the circuit costs associated with such subroutines.

Given these considerations, we expand on results from previous experiments in using Maximum Likelihood Estimation (MLE) to approximate the amplitude of a quantum state and provide empirical upper bounds on the current feasible circuit depths for AE on a superconducting quantum computer. Our results show that MLE using optimally-compiled circuits can currently outperform naive sampling for up to 3 Grover Iterations with a circuit depth of 131, which is higher than reported in other experimental results. This functional benchmark is one of many that will be continually monitored against current quantum hardware to measure the necessary progress towards quantum advantage.

I. INTRODUCTION

Many algorithms have been created which can provide a quadratic speedup of certain calculations [8], including those in finance like Monte Carlo simulations for derivative pricing and Value at Risk (VaR). These algorithms involve applying functions to one or more random variables and extracting the expected value of the function which is the quantity of interest. For example, finding the fair price for a European Call Option includes evaluating the payoff function based on the strike price and the distribution of the expected spot prices for the asset at maturity. In a quantum setting, this example would involve several steps, including loading the spot price distribution into a quantum state, computing the payoff function, and finally extracting the expected value from the amplitude of a quantum state [4][15][13][9].

While these steps leverage unique properties of quantum mechanics, it is only in the last step of extracting the expected value where the quadratic speedup is obtained. The retrieval of the expected value of these computations rely on Quantum Amplitude Estimation, which converges as $O(1/M)$ compared to the classical convergence rate of $O(1/\sqrt{M})$[8]. Therefore there is a motivation to perform Amplitude Estimation (AE) in an efficient and accurate way, and to benchmark its success on current hardware. Formally, the goal of AE is to provide an approximation of a prepared by an operator $\mathcal{A}$ such as in

$$\mathcal{A}\left|0_n\right\rangle\left|0\right\rangle = \sqrt{1-a}\left|\psi_0\right\rangle_n\left|0\right\rangle + \sqrt{a}\left|\psi_1\right\rangle_n\left|1\right\rangle \qquad (1)$$

In other words, there is some set of quantum operations $\mathcal{A}$ (e.g. the payoff function of a call option) that maps a quantity of interest (e.g. the expected value of the option) into a quantum state $\left|\psi_1\right\rangle\left|1\right\rangle$ and the goal is to find the amplitude of that state, $\sqrt{a}$. The canonical method [2] was to use Quantum Phase Estimation (QPE) to provide an estimate for the amplitude. This approach relied on using a controlled version of the grover iterate $\mathcal{Q}$, where $\mathcal{Q}$ is defined as

$$\mathcal{Q} = \mathcal{A}\mathcal{S}_0\mathcal{A}^\dagger\mathcal{S}_\psi$$

Here $\mathcal{S}_\psi = \mathcal{I} - 2\left|\psi_1\right\rangle_n\left\langle\psi_1\right|_n \bigotimes \left|1\right\rangle\left\langle 1\right|$ and $\mathcal{S}_0 = \mathcal{I} - 2\left|0\right\rangle_{n+1}\left\langle 0\right|_{n+1}$. In other words, $\mathcal{S}_\psi$ adds a negative sign to the $\left|\psi_1\right\rangle$ states and $\mathcal{S}_0$ reflects around the zero state.

By then performing Quantum Fourier Transform (QFT) on the control register of size m we would get an estimate for a.

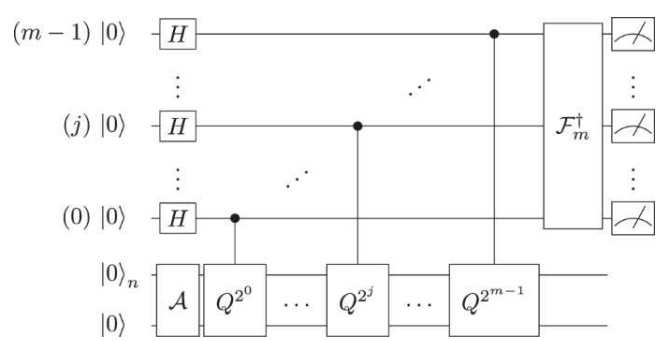

FIG. 1: Canonical algorithm for Amplitude Estimation by Quantum Phase Estimation.

This method has features that make it unsuitable for current Noisy Intermediate Scale Quantum devices. The

*Electronic address: scerto@deloitte.com

qubit topography of many quantum systems make controlled gates expensive and can quickly increase the depth of the circuit. Similarly, QFT requires expensive operations that are generally not practically implementable in current devices.

There has been active research into other ways to conduct AE, some even involving variational methods [10]. Most current methods involve non-controlled applications of $\mathcal{Q}$ and some classical post processing to create an estimate of a [6][14]. These methods rely on the fact that $\mathcal{A}\,|0_n\rangle = \cos\theta\,|\psi_0\rangle + \sin\theta|\psi_1\rangle$. If we can get an accurate estimate for θ, then $\sin^2\theta = a$. Included in these methods are the Maximum Likelihood Amplitude Estimation (MLAE) algorithm [14], which applies successive applications of the Grover Iterate $\mathcal{Q}$ and finds the θ that is most likely given the results. A benefit of MLAE is that it allows us to control the number of applications as well as the number of circuit evaluations for the $\mathcal{Q}$ oracle.

Given a number m_k of repetitions of $\mathcal{Q}$, the number of circuit evaluations N_k, and the number of times h_k that $|\psi_1\rangle\,|1\rangle$ was measured, the probability of measuring $|\psi_1\rangle\,|1\rangle$ is $P(m_k;a) = \sin^2\left((2m_k+1)\theta_a\right)$. The likelihood function is therefore

$$\mathcal{L}(h;a) = \prod_{k=0}^{M}[P(m_k;a)]^{h_k}[1-P(m_k;a)]^{N_k-h_k} \tag{2}$$

The number of repetitions of $\mathcal{Q}$ as well as the number of circuit evaluations per each repetition define the sequencing schedule for the algorithm. Different schedules have been proposed, including the Linearly Incremental Sequence (LIS), the Exponential Incremental Sequence (EIS), and the Power Law schedule. The experiments we conducted used the simple LIS schedule.

II. MOTIVATION FOR EXPERIMENT

Experimental results were recently published [5] that showed evidence of amplitude estimation outperforming naive sampling and low error rates on a state-of-the-art trapped ion quantum computer. It is known that the different types of quantum computers have advantages and disadvantages, and while some work has been published on benchmarking AE on superconducting hardware [11], there has been little published on actual circuit depths and their effects on amplitude estimation. While quantum engineers use high level languages to construct circuits, they are first compiled to solve the many constraints a particular quantum computer has, including qubit topography and actual basis gate implementations. This can have dramatic effects on the actual circuit depths, number and types of gates used, thereby altering the performance of the algorithm than what may be expected based on the non-compiled circuit. Therefore it is critical to continue to evaluate the status of current computers as well as the effect of circuit depth on their performance, on which we report in this paper.

III. AMPLITUDE ESTIMATION CIRCUITS USED IN MLE ALGORITHM

One can construct any arbitrary circuit to use for amplitude estimation, the only requirement being that there exists a well-defined quantum state from which we can benchmark the results. For our purposes, we wanted to design an experiment that is reproduce-able, scalable, and has advantageous properties related to the algorithm we are benchmarking. The circuits used in [5] share some of these properties. They originally were proposed to be used as a unary data loader, a method to load unit vectors and matrices into a quantum system. As it pertains to amplitude estimation and shown in [5], it also has nice properties for both the oracle $\mathcal{S}_\psi$ and diffuser $\mathcal{A}\mathcal{S}_0\mathcal{A}^\dagger$ of the Grover Iterate, requiring single Z gates for both $\mathcal{S}_\psi$ and $\mathcal{S}_0$.

The experiments previously mentioned used unary data loaders to load two unit vectors and compute their inner product in the amplitude of a quantum state that is to be estimated. It relied on the $\mathcal{RBS}$ gate which is a two qubit unitary matrix that is described as:

$$\mathcal{RBS}(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & \sin\theta & 0 \\ 0 & -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{3}$$

Based on the unit vectors, there is a method for finding θ. While we refer to [5] and [7] for more information on constructing the specific gates, we will briefly provide an example of using them. Given two unit vectors, $|a\rangle$ and $|b\rangle$, we can find angles through the procedure in [7] to construct the circuit below that will compute the square of their inner product $\langle a|b\rangle$ in the amplitude of the first qubit.

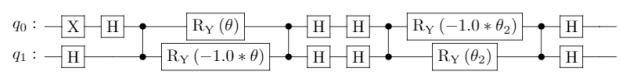

FIG. 2: Circuit for computing the inner product between two 2 dimensional unit vectors.

For instance,

$$|a\rangle = \begin{bmatrix}-0.96184207 & 0.27360523\end{bmatrix}$$
$$|b\rangle = \begin{bmatrix}0.96845237 & 0.24919873\end{bmatrix}$$
$$\langle a|b\rangle^2 = |01\rangle = .745314789774813$$

We used similar initial gate configurations in Qiskit as [5], however both the hardware we ran our circuits on as well as the compilers changed the gate configurations. Still, shown in Figure 3 and Figure 4 are the initial circuits for the 4 dimensional inner product circuits which represent $\mathcal{A}$ and $\mathcal{A}\mathcal{S}_0\mathcal{A}^\dagger\mathcal{S}_\psi\mathcal{A}$.

In the 4 dimensional case, $\mathcal{A}$ will load the states $a\,|0001\rangle + b\,|0010\rangle + c\,|0100\rangle + d\,|1000\rangle$ where a, b, c, d are

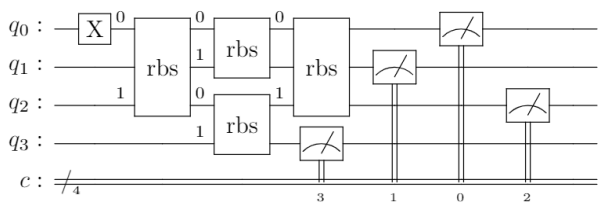

FIG. 3: Inner product estimation circuit for $\mathcal{A}$

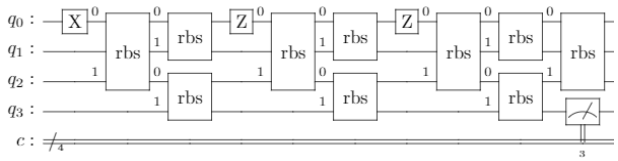

FIG. 4: Circuit for a single grover iteration of $\mathcal{A}\mathcal{S}_0\mathcal{A}^\dagger\mathcal{S}_\psi\mathcal{A}$

the amplitudes and $a^2 + b^2 + c^2 + d^2 = 1$. We are interested in the amplitude of a, the probability of the $|0001\rangle$ state which represents the squared inner product of the 4 dimensional unit vectors.

While we tested a trivial example on 2 qubits, the main result of our work was in estimating the inner product of 2 random 4 dimensional unit vectors. In this case, we constructed $\mathcal{A}$ and $\mathcal{A}\mathcal{S}_0\mathcal{A}^\dagger\mathcal{S}_\psi\mathcal{A}$ as shown above. The quantum computer we ran our circuits on took the high level circuits and compiled them to the allowable basis gates. We then used Qiskit and Pytket to optimize these compiled circuits as aggressively as allowed.

To estimate the amplitude of the first qubit which represents the inner product of the unit vectors, we added Grover Iterates from (0...7) where 0 is just $\mathcal{A}$. Once compiled based on the 3 different methodologies, we ran the circuit 500 times and used the results as input to our Maximum Likelihood Estimation algorithm. Therefore by using $m_k = 7$ of repetitions of $\mathcal{Q}$, we also used the previous results of $m_k = (0...6)$ for our calculation.

We generated random unit vectors whose inner product is uniformly distributed and instantiated our initial distribution for θ in the MLE algorithm to be a uniform distribution between $[0, \frac{\pi}{2}]$. For each set of unit vector pairs, we compared the MLE output and traditional sampling method to the exact inner product, giving us our estimate of relative error.

IV. IBM NAIROBI SPECIFICATIONS

The experiments conducted were done using the IBM Nairobi superconducting computer. It has 7 qubits with a quantum volume of 32. The exact specifications are detailed in the specifications table.

IBM NAIROBI SPECIFICATIONS

Processor Type	*Falcon r5.11H*
Version	*1.0.7*
Avg CNOT Error	*9.006e-3*
Avg Readout Error	*3.546e-2*
Avg T1	*108.2 us*
Avg T2	*59.4 us*

V. COMPARISON OF OPTIMIZED CIRCUITS

The RBS gates outlined above have significant depths when compiled on IBM hardware. In fact, they can reach a depth of over 500 for 7 Grover Iterations. Knowing the need to reduce these depths and optimize the circuits as much as possible, we explored two well known open source compilers provided by Qiskit [1] and Pytket [12]. There are many ways to optimally compile a quantum circuit, none of which are trivial. Considerations need to be made for the goal of the algorithm, the specific hardware, and the types of gates required along with their typical error rates.

Both Qiskit and Pytket have a similar set of compilation passes, first translating the high level circuit into the required basis gates of the hardware, then attempting to optimize the circuit to reduce depth and noise. As referenced in [12], Pytket's optimization compiler falls into two broad categories: peephole optimizations and macroscopic optimizations. Peephole optimizations are similar to those in classical settings, where a sliding window looks for local patterns that can be substituted with equivalent sub-circuits with lower gate counts or depth. Macroscopic optimization looks for non-local structures that can be synthesized to increase parallelization or fewer gates, potentially leading to another pass of peephole optimization. We refer to [12] for more detailed information.

In figure 6 we show a sample of circuits for certain 4 dimensional vectors with no Grover iterations. Both Qiskit and Pytket substantially reduced the two-qubit interactions, gate counts, and overall circuit depth. In figure 5 we detail the number of gates and circuit depth per Grover iteration, as well as the total (sum of all 7 sub-circuits) counts per gate type. We can see that both Qiskit and Pytket reduced the number of gates and the circuit depth; the circuit depth was reduced by almost half and was a substantial improvement over the standard compilation procedure.

Furthermore, both found the minimum number of CNOT gates that could be used, while the standard compiler used more than double. We leave it to further ex-

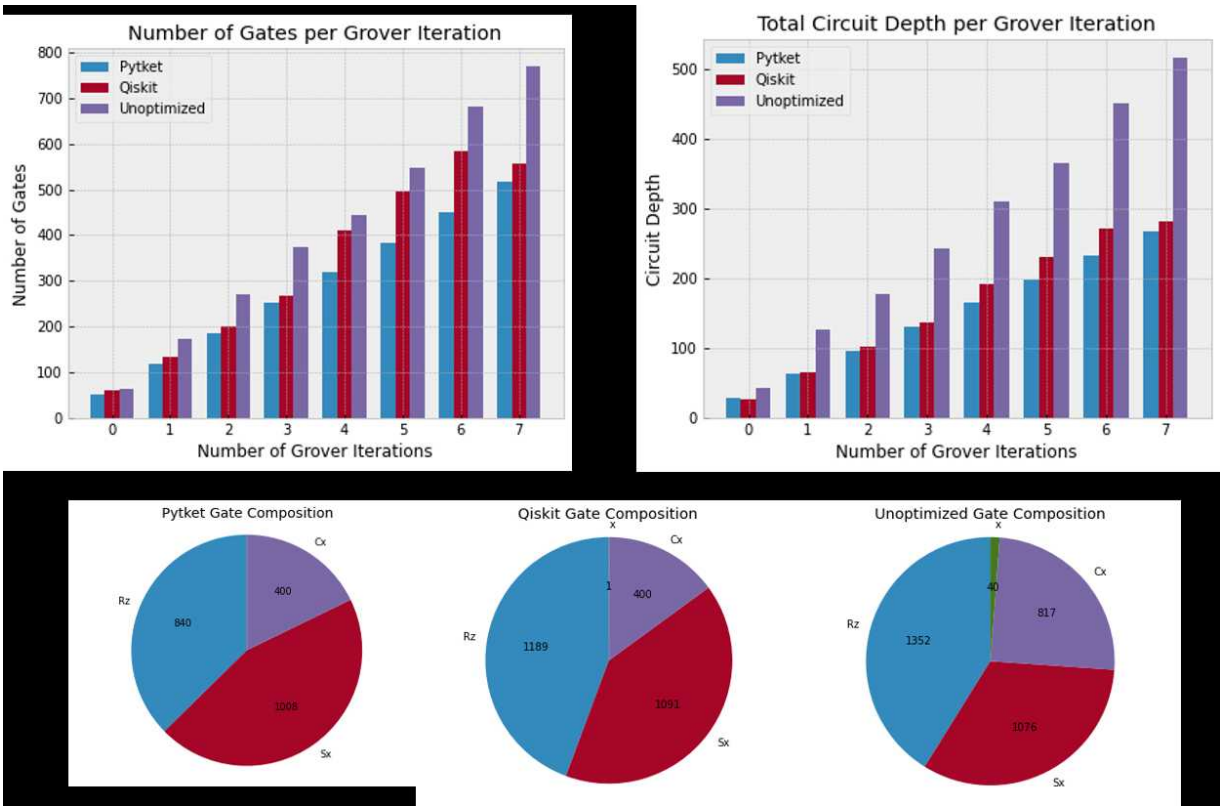

FIG. 5: Depth, gate count, and gate composition of various compilation methods

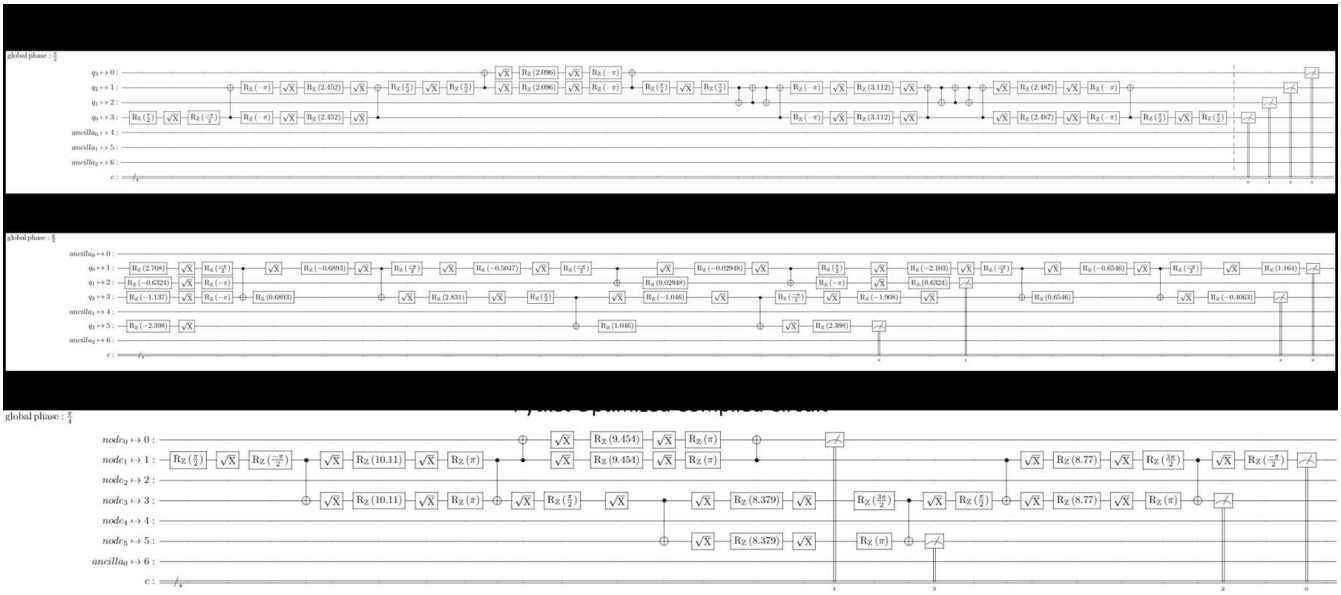

FIG. 6: Comparison of $\mathcal{A}$ circuits.

periments to benchmark the different ways to optimize quantum circuits; the results we present are not direct comparisons between either approaches. If we were to do so, we would choose a variety of metrics, algorithms, and circuit types to quantitatively measure them. Here we are simply showing in this specific case both are worthwhile to use versus the standard compiler.

VI. MINIMAL DEPTH AMPLITUDE ESTIMATION RESULTS

Here we will review the results of estimating the amplitude of random unit vectors for both 2 and 4 dimensions. We used a linear sequencing schedule for MLE, of 500 shots each up to 7 Grover Iterations. The experiment was repeated 50 times each, and the results averaged. For naive sampling, we ran the circuits for up to 3000 shots in increments of 500.

For the 2 qubit experiments, both the ordinarily-compiled circuits as well as the optimized circuits outperformed naive sampling. Interestingly, the larger number of shots for naive sampling did not reduce the error, which is most likely due to the fact that the inherent noise found on current hardware prohibits better amplitude estimates. As shown in Figure 7, The lowest recorded mean error of .0017 was after 7 Grover iterations, although after 3 Grover iterations the decrease in error became minimal.

While the 2 qubit experiment is trivially small, it demonstrated that MLE does outperform simple averaging of samples and after even 1 Grover iteration the error decreased by half. It also showed the impact of reducing circuit depth, complexity, and optimizing it for a particular quantum computer.

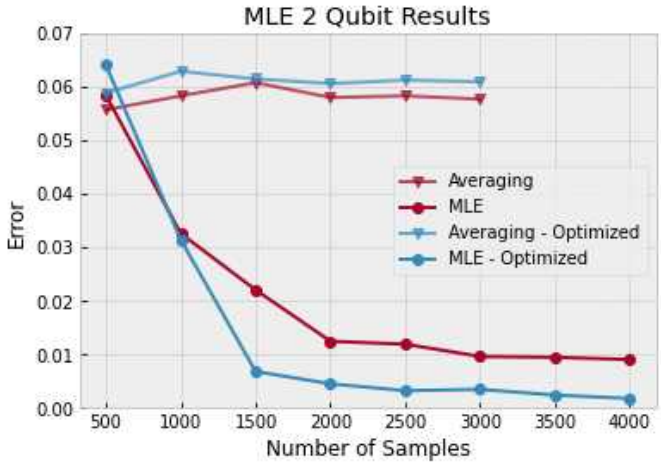

FIG. 7: 2 qubit AE results for ordinary transpiled and optimized depth circuits. The circles represent the maximum number of Grover Iterations used in MLE, with 500 shots per each iteration.

The 4 qubit results are more significant as the experiment still showed MLE outperforming naive sampling with much larger circuit depths and 2-qubit gates. The circuit compiler provided by both Qiskit and Pytket reduced the error rates for both simple averaging and MLAE, with Pytket providing a shallower circuit with lower mean error. Even with an optimal circuit layout, the depths reached a maximum of around 300 and the errors did deteriorate after 3 Grover iterations caused by decoherence of the qubits and compounded errors throughout the circuit. Both Qiskit and Pytket have different levels of circuit compilers, ranging from (0,1,2,3) for Qiskit and (0,1,2) for Pytket. Each of these compilation levels differ in their aggressiveness for optimizing the ciruits and reducing the depth. We used the most aggressive compilers from each and the results are shown in Figure 8.

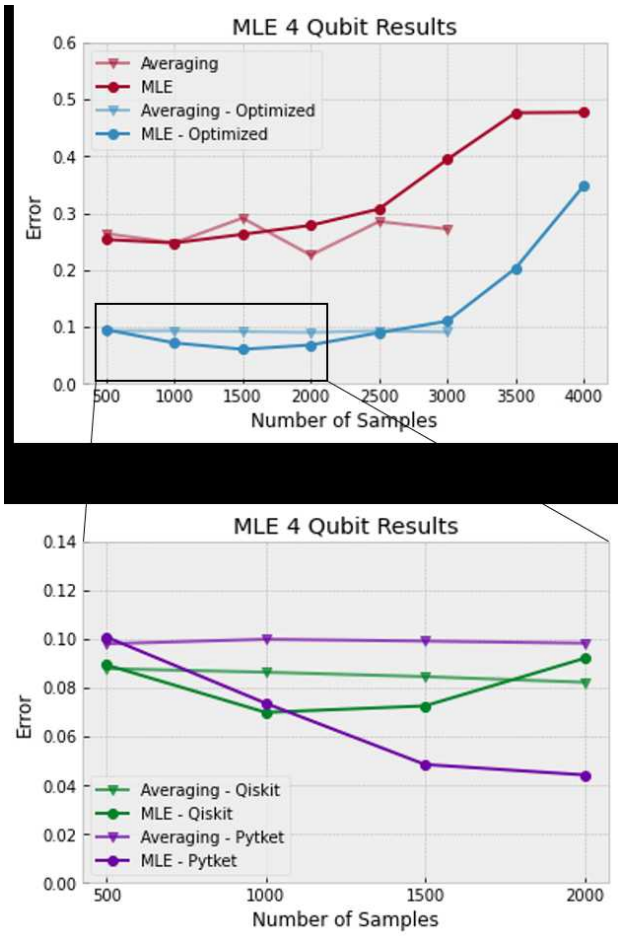

FIG. 8: 4 qubit AE results for ordinary compiled and optimized depth circuits. The optimized circuits were done using level 2 optimization provided by Pytket and level 3 optimization by Qiskit. The best results achieved in our experiments were found after 3 Grover iterations on the Pytket optimally-compiled circuits, achieving a mean error of .0442.

It is worthy to note that while Pytket seemed more robust to noise in the higher number of Grover Iterations, this could be the result of several causes. We used

50 random vectors for each experiment, and so the exact distribution of this small sample size could account for the discrepancy. Qiskit had a lower sampling error, which we also believe could have been the result of noise in the quantum system as well as the random vectors. Nonetheless, both offer substantial improvement over non-optimized circuits and show how significant an impact a well-compiled circuit performs over an ordinarily-compiled one. Even the results from the naive averaging of the non-optimized circuits had a much larger variance than the optimized circuits from both Qiskit and Pytket, showing that the compilation provided by both substantially reduces unwanted noise and stabilizes the results.

Compared to [5], our optimized circuits had higher depths than the experiments performed on a trapped ion computer, which reported two-qubit gates up to ninety two and a maximum circuit depth of sixty two. The depths reported here are much larger compared to previous experiments as we were able to achieve a minimum mean error of around .04 with circuit depths of 131 for 3 Grover iterations. In fact, while the circuits constructed were minimal with respect to IBM's superconducting computers, they are some of the deepest circuits reported thus far that still obtained a significant reduction in error while performing MLAE. That said, recent work has determined that the threshold for quantum advantage in certain Monte Carlo simulations requires a t-depth of around 54 million [3], and so while current hardware is out of reach for such performance, we hope that functional benchmark tests such as this experiment continue to evaluate the maturity of current systems in the pursuit of such an advantage.

VII. CONCLUSION

In this paper we have expanded on previous work [5] in using circuits that compute the inner product between unit vectors to benchmark the Maximum Likelihood Amplitude Estimation algorithm. While we have shown that both the circuit optimizers by Qiskit and Pytket work very well in reducing noise and increasing the allowable number of Grover iterations we can perform in the algorithm, we have more importantly provided an empirical upper bound on the maximum allowable depth for AE on current hardware, a metric that will be will closely monitored as the industry progresses.

VIII. DISCLAIMER

About Deloitte: Deloitte refers to one or more of Deloitte Touche Tohmatsu Limited ("DTTL"), its global network of member firms, and their related entities (collectively, the "Deloitte organization"). DTTL (also referred to as "Deloitte Global") and each of its member firms and related entities are legally separate and independent entities, which cannot obligate or bind each other in respect of third parties. DTTL and each DTTL member firm and related entity is liable only for its own acts and omissions, and not those of each other. DTTL does not provide services to clients. Please see www.deloitte.com/about to learn more.

Deloitte is a leading global provider of audit and assurance, consulting, financial advisory, risk advisory, tax and related services. Our global network of member firms and related entities in more than 150 countries and territories (collectively, the "Deloitte organization") serves four out of five Fortune Global 500® companies. Learn how Deloitte's approximately 330,000 people make an impact that matters at www.deloitte.com. This communication contains general information only, and none of Deloitte Touche Tohmatsu Limited ("DTTL"), its global network of member firms or their related entities (collectively, the "Deloitte organization") is, by means of this communication, rendering professional advice or services. Before making any decision or taking any action that may affect your finances or your business, you should consult a qualified professional adviser. No representations, warranties or undertakings (express or implied) are given as to the accuracy or completeness of the information in this communication, and none of DTTL, its member firms, related entities, employees or agents shall be liable or responsible for any loss or damage whatsoever arising directly or indirectly in connection with any person relying on this communication. Copyright © 2021. For information contact Deloitte Global.

[1] Qiskit: An open-source framework for quantum computing, 2021.

[2] Gilles Brassard, Peter Høyer, Michele Mosca, and Alain Tapp. Quantum amplitude amplification and estimation. *Quantum Computation and Information*, page 53–74, 2002.

[3] Shouvanik Chakrabarti, Rajiv Krishnakumar, Guglielmo Mazzola, Nikitas Stamatopoulos, Stefan Woerner, and William J. Zeng. A threshold for quantum advantage in derivative pricing. *Quantum*, 5:463, Jun 2021.

[4] Daniel J Egger, Ricardo García Gutiérrez, Jordie Cahue Mestre, and Stefan Woerner. Credit risk analysis using quantum computers. *IEEE Transactions on Computers*, 2020.

[5] Tudor Giurgica-Tiron, Sonika Johri, Iordanis Kerenidis, Jason Nguyen, Neal Pisenti, Anupam Prakash, Ksenia Sosnova, Ken Wright, and William Zeng. Low depth amplitude estimation on a trapped ion quantum computer, 2021.

[6] Dmitry Grinko, Julien Gacon, Christa Zoufal, and Stefan

Woerner. Iterative quantum amplitude estimation. *npj Quantum Information*, 7(1):1–6, 2021.

[7] Sonika Johri, Shantanu Debnath, Avinash Mocherla, Alexandros Singh, Anupam Prakash, Jungsang Kim, and Iordanis Kerenidis. Nearest centroid classification on a trapped ion quantum computer, 2020.

[8] Ashley Montanaro. Quantum speedup of monte carlo methods. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 471(2181):20150301, Sep 2015.

[9] Román Orús, Samuel Mugel, and Enrique Lizaso. Quantum computing for finance: overview and prospects. *Reviews in Physics*, page 100028, 2019.

[10] Kirill Plekhanov, Matthias Rosenkranz, Mattia Fiorentini, and Michael Lubasch. Variational quantum amplitude estimation, 2021.

[11] Pooja Rao, Kwangmin Yu, Hyunkyung Lim, Dasol Jin, and Deokkyu Choi. Quantum amplitude estimation algorithms on ibm quantum devices, 2020.

[12] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. t—ket⟩: a retargetable compiler for nisq devices. *Quantum Science and Technology*, 6(1):014003, Nov 2020.

[13] Nikitas Stamatopoulos, Daniel J Egger, Yue Sun, Christa Zoufal, Raban Iten, Ning Shen, and Stefan Woerner. Option pricing using quantum computers. *Quantum*, 4:291, 2020.

[14] Yohichi Suzuki, Shumpei Uno, Rudy Raymond, Tomoki Tanaka, Tamiya Onodera, and Naoki Yamamoto. Amplitude estimation without phase estimation. *Quantum Information Processing*, 19(2):75, 2020.

[15] Stefan Woerner and Daniel J Egger. Quantum risk analysis. *npj Quantum Information*, 5(1):1–8, 2019.